UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

FOOTBALL MONK

A PROJECT REPORT

Submitted by **Ashutosh Dash**

(26MCD10089)

in partial fulfillment for the award of the degree of

Masters of Computer Applications

IN

Data science

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

2026-28

UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Chandigarh University
2026 - 28

BONAFIDE CERTIFICATE

Certified that this project report titled "Football Monk" is the bonafide work of Ashutosh Dash (UID: 26MCD10089) who carried out the project work under my/our supervision in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications at Chandigarh University.

SIGNATURE

Name: Ashutosh Dash

UID: 26MCD10089

Class: MCD - 2/A

SIGNATURE

Name: Ms.Divyanshi Sharma

Designation: Asst. Professor

E-Code: E17438

ACKNOWLEDGEMENT

The successful completion of this mini project would not have been possible without the guidance, support, and encouragement of several individuals and institutions. It is a matter of great pleasure and privilege for me to express my sincere gratitude to everyone who contributed, directly or indirectly, to the completion of this project titled "Football Monk – Away-Trip Route Planner using Dijkstra's Algorithm." Designing and implementing a route planner based on a shortest-path algorithm was a valuable learning experience in Design and Analysis of Algorithms, and I am grateful for the support I received at every stage.

First and foremost, I would like to thank Chandigarh University for giving me the opportunity to undertake this project as part of my academic curriculum for the Master of Computer Applications (Data Science) program. The university's learning environment, modern facilities, and resources helped me turn the theoretical concepts of graphs, greedy strategy, priority queues, and complexity analysis into a working program.

I extend my deepest appreciation to my project supervisor, : Ms.Divyanshi Sharma, Assistant Professor, University Institute of Computing, Chandigarh University, for constant guidance, constructive suggestions, and motivation throughout the project. Their expertise and willingness to clear my doubts helped me with algorithm selection, correctness and complexity analysis, and validation of the results against the Floyd–Warshall algorithm. Their encouragement inspired me to overcome challenges and improve the quality of this work.

I am also thankful to the Head of the Department, the faculty members, and the staff of the University Institute of Computing for their cooperation and timely feedback. I would also like to acknowledge the teaching and non-teaching staff who provided the infrastructure and resources needed for development, testing, and documentation.

Finally, I thank my friends and classmates for their encouragement, suggestions, and moral support. Their discussions and feedback helped me improve the menu design and test cases, and kept me motivated during the more demanding phases of implementation.

1. AIM

The aim of this project is to design and develop Football Monk, a Python console application that plans away-trip routes for football supporters. The road network between football cities is modelled as a weighted, undirected graph, and Dijkstra's single-source shortest-path algorithm computes the minimum travel distance from the fans' home city to the stadium of every away fixture. For each stadium the planner reports the shortest distance, an estimated travel time, an estimated cost and the route in order.

The planner stores the graph as an adjacency list, uses a binary min-heap (Python's heapq module) as the priority queue and records a predecessor for every city so that the complete route can be reconstructed. The sample network contains 15 cities and 22 roads, and the results were validated against the Floyd–Warshall all-pairs algorithm: all 225 source–destination distances matched. The running time of the implementation is O((V + E) log V) and its space requirement is O(V + E). The project demonstrates the greedy strategy, priority queues and complexity analysis taught in the DAA course through a practical, easy-to-run program, and is part of the wider Football Monk theme, which started with the Football Jersey Management System built in Advanced Internet Programming.

2. OBJECTIVES AND PROBLEM DEFINITION

2.1 Problem Definition

Football clubs play half of their league matches away from home, and thousands of supporters travel every weekend to cheer for their team. For the fans of a club based in one city, the distance to each opponent's stadium differs greatly, and the best route is rarely the most obvious one. Choosing roads by guesswork wastes time and money, and the problem becomes harder as the number of cities and roads grows.

Formally, given a weighted undirected graph G = (V, E) in which each vertex is a football city and each edge is a road with a non-negative distance, and given a home city s, find the shortest distance and the corresponding route from s to the stadium city of every away fixture, and present the results to the user with estimated travel time and cost.

2.2 Objectives

- **Modelling:** Represent the road network of football cities as a weighted graph.
- **Algorithm:** Implement Dijkstra's algorithm correctly with a priority queue.
- **Route reconstruction:** Show not only the distance but also the exact route to every stadium.
- **Usability:** Offer a simple menu so that any city can be chosen as the home city.
- **Analysis:** Validate the output and analyse the time and space complexity of the solution.

2.3 Scope of the Project

The scope covers the design, implementation, testing and analysis of a console-based route planner in Python. A graphical map, live traffic data and real road APIs are outside the scope and are listed under future work.

Feature	Description
Graph modelling	Represents cities and roads as a weighted undirected graph (adjacency list)
Shortest-path search	Dijkstra's algorithm with a binary min-heap and a predecessor array
Route reconstruction	Rebuilds and displays the route to every away stadium
Time and cost estimate	Distance divided by average speed; distance multiplied by cost per km
Interactive menu	Choose or change the home city, view all trips or plan a single trip
Validation	Compares results with Floyd–Warshall and runs menu and input test cases
Out of scope	Graphical map, live traffic data and real road APIs (future work)

Table 1. Major functional components of Football Monk.

2.4 Project Planning

Project planning involves the following phases:

- **Problem analysis:** Understanding the away-trip problem and mapping it to a single-source shortest-path problem.
- **Data design:** Collecting sample cities, stadiums and approximate road distances.
- **Algorithm design:** Choosing the graph representation and the priority queue, and writing the pseudocode.
- **Implementation:** Coding the graph builder, Dijkstra function, path reconstruction and menu in Python.
- **Testing and analysis:** Validating results against Floyd–Warshall, testing user inputs and analysing time and space complexity.

Phase	Activity	Duration
Phase 1	Problem analysis and data collection	Week 1

Phase	Activity	Duration
Phase 2	Algorithm design, pseudocode and graph model	Week 1 – 2
Phase 3	Implementation of Dijkstra, menu and output	Week 2 – 3
Phase 4	Testing, complexity analysis and report writing	Week 3

Table 2. Timeline of the project.

2.5 Users and Relevance

The primary users are football supporters and supporters' clubs who organise coach or car trips to away fixtures. Travel organisers also benefit because the shortest distance directly gives the fuel cost and the travel time that must be planned for. These users need a tool that is simple to run, shows distance and route to every stadium at once, and can be updated easily when roads or fixtures change.

Navigation and route optimisation are among the most widely used applications of graph algorithms today, from map applications to logistics and ride-hailing. Rising fuel prices and the focus on reducing carbon emissions make shorter, more efficient trips important for large groups of travellers. Understanding how a shortest-path algorithm works, rather than treating map software as a black box, is therefore a relevant skill for a computer applications student.

3. PROGRAMMING LANGUAGES AND TECHNOLOGIES USED

Python is the primary language for the graph, the algorithm, the output and the interactive menu. Only the Python standard library is used (no networkx or map APIs) so that the shortest-path algorithm itself is implemented and understood, and the program runs in a standard console inside VS Code.

Layer	Technology	Purpose
Language	Python 3.8 or newer	Implementation of the graph, algorithm and menu
Data structure	dict and list (adjacency list)	Stores the road network
Priority queue	heapq (binary min-heap)	Selects the nearest unvisited city in O(log V)

Layer	Technology	Purpose
Interface	Console (standard input/output)	Interactive menu, no external libraries needed
IDE	Visual Studio Code	Writing and running the program

Table 3. Technologies used in Football Monk.

4. SYSTEM DESIGN AND ARCHITECTURE

4.1 System Architecture

Football Monk is organised as a small set of cooperating functions. The data lists (ROADS and STADIUMS) feed build_graph(), which creates the adjacency list; dijkstra() computes distances and predecessors; get_path() rebuilds routes; the printing helpers format the output; and main() provides the menu.

The program first builds the adjacency list from the road list and asks the user for the home city. Dijkstra's algorithm is then run once from that city. It keeps a distance for each city, initially infinity except for the source, and a min-heap of (distance, city) pairs. The nearest unsettled city is repeatedly removed from the heap, and each road leaving it is relaxed: if reaching a neighbour through this city is shorter than the best known distance, the distance and predecessor are updated and the neighbour is pushed onto the heap. When the heap is empty, every city holds its shortest distance, and the routes are printed by following the predecessors backwards.

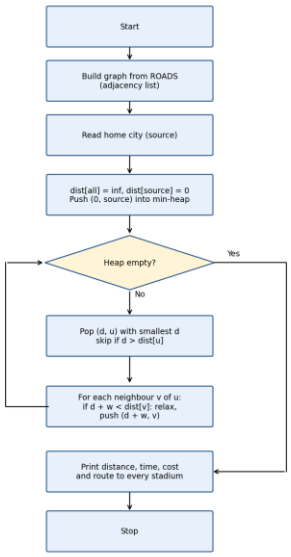


FIGURE 1 — FLOWCHART OF THE AWAY-TRIP ROUTE PLANNER

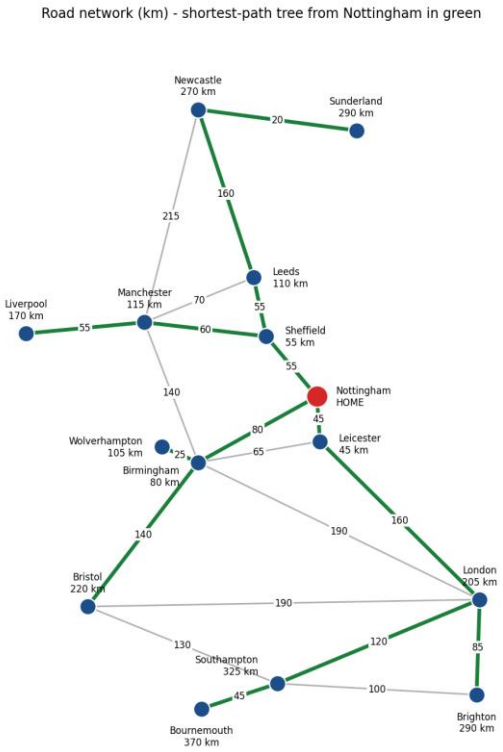


FIGURE 2 — SAMPLE ROAD NETWORK OF FOOTBALL CITIES (EDGE LABELS IN KM)

4.2 Module Description

Function	Main responsibility
build_graph(roads)	Converts the list of roads into an adjacency list; every road is stored in both directions because the graph is undirected
dijkstra(graph, source)	Implements the algorithm with heapq; stale heap entries (d > dist[u]) are skipped instead of deleted (lazy deletion)
get_path(prev, target)	Walks the predecessor chain from the target back to the source and reverses it to give the route in travel order
hours_text(km)	Converts a distance into travel time at the average speed (70 km/h)
print_all_trips(), print_one_trip()	Print distance, travel time, estimated cost (distance multiplied by cost per km) and the route
pick_city(), main()	Accept a city by number or by name (case-insensitive), repeat on invalid input and provide the four-option menu

Table 4. Main functions and their responsibilities.

4.3 Design Approach

Feature selection. Features were chosen according to what a supporter needs when planning an away trip and what can be built reliably in the available time: a choice of home city, a table of the shortest trips to all stadiums sorted from nearest to farthest, a detailed plan for a single stadium, estimated time and cost, and the ability to change the home city without restarting the program.

Design constraints. Only the Python standard library may be used; edge weights must be non-negative, which is a requirement of Dijkstra's algorithm; the road network is an undirected graph with approximate sample distances in kilometres; and the program must run in a standard console inside VS Code.

Methodology. The system is implemented bottom-up: the data lists first, then build_graph(), then dijkstra() and get_path(), then the printing helpers and finally the menu in main(). Each function was tested on its own before the next was added, and the complete algorithm was finally validated against Floyd–Warshall on every source–destination pair.

4.4 Data Design

Data structure	Type in Python	Role
ROADS	list of (city, city, km)	Raw road data entered by the user
STADIUMS	dict: city → (stadium, club)	Maps each city to its home stadium and club
graph	dict: city → list of (neighbour, km)	Adjacency list of the weighted undirected graph
dist	dict: city → number	Best known distance from the source (infinity at start)
prev	dict: city → city or None	Predecessor on the shortest path, used to rebuild routes
pq	list used as a heap (heapq)	Min-heap of (distance, city) pairs

Table 5. Data structures used.

5. ALGORITHMS AND PSEUDOCODE

Dijkstra's algorithm was selected after comparing the common shortest-path algorithms, because the planner must find distances from one home city to many stadiums at the same time on a network that never has negative distances.

5.1 Shortest-Path Algorithms Considered

- **Breadth-first search (BFS):** Finds the path with the fewest edges, but ignores edge weights, so it cannot find the shortest distance on a road network.
- **Dijkstra's algorithm:** Finds the shortest paths from one source to all vertices when all weights are non-negative. It is efficient and is the basis of many routing systems.
- **Bellman–Ford algorithm:** Handles negative edge weights and detects negative cycles, but runs in O(VE), which is slower for road networks that never have negative distances.
- **Floyd–Warshall algorithm:** Computes shortest paths between all pairs of vertices in $O(V^3)$. It is simple but too slow for large graphs; in this project it is used only to validate the results.
- **A* search:** Extends Dijkstra with a heuristic to reach a single target faster, but needs coordinates or a good estimate of the remaining distance.

Algorithm	Negative weights	Time complexity	Best use
BFS	Not applicable	O(V + E)	Unweighted graphs
Dijkstra (heap)	No	O((V + E) log V)	Single source, non-negative weights
Bellman–Ford	Yes	O(V·E)	Graphs with negative weights
Floyd–Warshall	Yes (no negative cycle)	$O(V^3)$	All pairs, small graphs
A*	No	Depends on heuristic	Single source to single target

Table 6. Comparison of shortest-path algorithms.

Dijkstra's algorithm with a binary heap is therefore the most suitable choice: it is faster than Bellman–Ford and Floyd–Warshall for this problem, and, unlike A*, it produces the shortest distance to every city in a single run without needing map coordinates.

5.2 Dijkstra's Algorithm

A greedy algorithm that repeatedly settles the nearest unsettled city and relaxes the roads leaving it, using a min-heap of (distance, city) pairs and a predecessor for every city.

Pseudocode:

```
DIJKSTRA(G, s):

    for each vertex v in G:  dist[v] = INFINITY;  prev[v] = NIL

    dist[s] = 0
```

```
PQ = min-heap containing (0, s)
while PQ is not empty:
    (d, u) = EXTRACT-MIN(PQ)
    if d > dist[u]:  continue            // stale entry
    for each edge (u, v) with weight w:
        if d + w < dist[v]:               // relaxation
            dist[v] = d + w;  prev[v] = u
            INSERT(PQ, (dist[v], v))
return dist, prev
```

5.3 Complexity Analysis

Let V be the number of cities and E the number of roads. Every vertex is settled once, and every edge is relaxed at most twice (once from each end, because the graph is undirected). Each insertion into or removal from the binary heap costs O(log V) (more precisely O(log E), which is the same order since $E \leq V^2$). Because a vertex may be inserted more than once (lazy deletion), the heap holds at most O(E) entries.

Quantity	Result	Reason
Building the adjacency list	O(V + E)	Each road is added to two lists
Heap operations	O(E log V)	At most O(E) pushes and pops, each O(log V)
Total time of Dijkstra	O((V + E) log V)	Dominated by heap operations
Space	O(V + E)	Adjacency list, dist, prev and heap
Path reconstruction (one stadium)	O(V)	Follow prev pointers back to the source
Floyd–Warshall (validation only)	$O(V^3)$	Three nested loops over all vertices

Table 7. Complexity analysis.

Correctness (greedy-choice property): when a vertex u is removed from the heap with the smallest tentative distance, no shorter path to u can exist, because every other path must leave the settled set through a vertex with a distance at least as large, and all edge weights are non-negative. Hence the distance of u is final. This argument fails for negative weights, which is why the planner only accepts non-negative distances.

6. IMPLEMENTATION

6.1 Source Code

The source file is away_trip_planner.py and it runs with the command python away_trip_planner.py.

```
import heapq

import math


try:

    import matplotlib.pyplot as plt

    from matplotlib import cm

    from matplotlib.colors import Normalize

    HAS_MPL = True

except ImportError:

    HAS_MPL = False



ROADS = [

    ("London", "Brighton", 85),

    ("London", "Southampton", 120),

    ("London", "Leicester", 160),

    ("London", "Birmingham", 190),

    ("London", "Bristol", 190),

    ("Brighton", "Southampton", 100),

    ("Southampton", "Bournemouth", 45),

    ("Southampton", "Bristol", 130),

    ("Bristol", "Birmingham", 140),

    ("Birmingham", "Wolverhampton", 25),

    ("Birmingham", "Leicester", 65),

    ("Birmingham", "Nottingham", 80),

    ("Birmingham", "Manchester", 140),

    ("Leicester", "Nottingham", 45),

    ("Nottingham", "Sheffield", 55),

    ("Sheffield", "Manchester", 60),

    ("Sheffield", "Leeds", 55),

    ("Manchester", "Liverpool", 55),

    ("Manchester", "Leeds", 70),
```

```
    ("Leeds", "Newcastle", 160),
    ("Newcastle", "Sunderland", 20),
    ("Manchester", "Newcastle", 215),
]

STADIUMS = {  # city -> (stadium, club)
    "London": ("Emirates Stadium", "Arsenal"),
    "Brighton": ("Amex Stadium", "Brighton"),
    "Southampton": ("St Mary's Stadium", "Southampton"),
    "Bournemouth": ("Vitality Stadium", "Bournemouth"),
    "Bristol": ("Ashton Gate", "Bristol City"),
    "Birmingham": ("Villa Park", "Aston Villa"),
    "Wolverhampton": ("Molineux", "Wolves"),
    "Leicester": ("King Power Stadium", "Leicester City"),
    "Nottingham": ("City Ground", "Nottingham Forest"),
    "Sheffield": ("Bramall Lane", "Sheffield United"),
    "Manchester": ("Old Trafford", "Manchester United"),
    "Liverpool": ("Anfield", "Liverpool"),
    "Leeds": ("Elland Road", "Leeds United"),
    "Newcastle": ("St James' Park", "Newcastle United"),
    "Sunderland": ("Stadium of Light", "Sunderland"),
}

COORDS = {
    "London": (-0.12, 51.50),
    "Brighton": (-0.14, 50.82),
    "Southampton": (-1.40, 50.90),
    "Bournemouth": (-1.88, 50.72),
    "Bristol": (-2.59, 51.45),
    "Birmingham": (-1.90, 52.48),
    "Wolverhampton": (-2.13, 52.59),
    "Leicester": (-1.13, 52.64),
    "Nottingham": (-1.15, 52.95),
    "Sheffield": (-1.47, 53.38),
    "Manchester": (-2.24, 53.48),
    "Liverpool": (-2.98, 53.41),
    "Leeds": (-1.55, 53.80),
```

```
    "Newcastle": (-1.61, 54.98),
    "Sunderland": (-1.38, 54.91),
}

AVG_SPEED_KMH = 70
COST_PER_KM = 0.12


def build_graph(roads):
    """Undirected weighted graph as an adjacency dict."""
    graph = {}
    for a, b, km in roads:
        graph.setdefault(a, []).append((b, km))
        graph.setdefault(b, []).append((a, km))
    return graph


def dijkstra(graph, source):
    """Return (dist, prev). dist[city] = shortest km from source;
    prev[city] = previous city on that shortest path."""
    dist = {city: float("inf") for city in graph}
    prev = {city: None for city in graph}
    dist[source] = 0
    pq = [(0, source)]

    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                prev[v] = u
                heapq.heappush(pq, (nd, v))
    return dist, prev
```

```
for a, b, km in ROADS:
    (x1, y1), (x2, y2) = COORDS[a], COORDS[b]
    on_route = frozenset((a, b)) in highlight
    ax.plot([x1, x2], [y1, y2],
            color="#d62728" if on_route else "#b0b0b0",
            linewidth=3.2 if on_route else 1.2,
            zorder=2 if on_route else 1,
            solid_capstyle="round")
    ax.text((x1 + x2) / 2, (y1 + y2) / 2, str(km), fontsize=7,
            ha="center", va="center", zorder=3,
            color="#d62728" if on_route else "#555555",
            bbox=dict(boxstyle="round,pad=0.15", fc="white",
                      ec="none", alpha=0.85))


reachable = [d for d in dist.values() if d != float("inf")]
norm = Normalize(vmin=0, vmax=max(reachable))
cmap = cm.RdYlGn_r
for city, (x, y) in COORDS.items():
    is_home = city == home
    is_dest = city == dest
    ax.scatter(x, y, s=330 if (is_home or is_dest) else 190,
               marker="*" if is_home else "o",
               color="gold" if is_home else cmap(norm(dist[city])),
               edgecolors="black" if (is_home or is_dest) else "#333333",
               linewidths=2 if is_dest else 1, zorder=4)
    club = STADIUMS[city][1]
    ax.annotate(f"{city}\n{club}" if not is_home else f"{city} (HOME)\n{club}",
                (x, y), xytext=(10, 6), textcoords="offset points",
                fontsize=8, fontweight="bold" if (is_home or is_dest) else "normal",
                zorder=5)


sm = cm.ScalarMappable(norm=norm, cmap=cmap)
cbar = fig.colorbar(sm, ax=ax, shrink=0.55, pad=0.02)
```

```
    cbar.set_label(f"Shortest distance from {home} (km)")

    if dest is not None:
        title = (f"{home}  ->  {dest}   |   {dist[dest]} km, "
                 f"{hours_text(dist[dest])}, cost {dist[dest] * COST_PER_KM:.2f}")
    else:
        title = f"Shortest-path tree from {home} (red = best roads)"
    ax.set_title(title, fontsize=12, fontweight="bold")
    ax.set_xlim(-3.9, 0.9)
    ax.set_ylim(50.4, 55.3)
    ax.set_xticks([])
    ax.set_yticks([])
    fig.tight_layout()

    if save_as:
        fig.savefig(save_as, dpi=150)
        print(f"Map saved to {save_as}")
    plt.show()
def hours_text(km):
    minutes = round(km / AVG_SPEED_KMH * 60)
    return f"{minutes // 60}h {minutes % 60:02d}m"


def print_all_trips(home, dist, prev):
    print(f"\nAWAY TRIPS FROM {home.upper()}  (sorted nearest -> farthest)")
    print("=" * 96)
    print(f"{'Club':<19}{'Stadium':<21}{'City':<14}{'km':>5}{'Time':>9}{'Cost':>8}  Route")
    print("-" * 96)
    rows = sorted((c for c in STADIUMS if c != home), key=lambda c: dist[c])
    for city in rows:
        stadium, club = STADIUMS[city]
        km = dist[city]
        route = " > ".join(get_path(prev, city))
        print(f"{club:<19}{stadium:<21}{city:<14}{km:>5}{hours_text(km):>9}"
              f"{km * COST_PER_KM:>8.2f}  {route}")
    print("-" * 96)
```

```
def print_one_trip(home, dest, dist, prev):
    stadium, club = STADIUMS[dest]
    path = get_path(prev, dest)
    print(f"\n{club} - {stadium}, {dest}")
    print(f"Shortest distance : {dist[dest]} km")
    print(f"Travel time       : {hours_text(dist[dest])} at {AVG_SPEED_KMH} km/h")
    print(f"Estimated cost    : {dist[dest] * COST_PER_KM:.2f}")
    print("Route             : " + " > ".join(path))


def pick_city(prompt, cities):
    for i, c in enumerate(cities, 1):
        print(f"  {i:>2}. {c}")
    while True:
        raw = input(prompt).strip()
        if raw.isdigit() and 1 <= int(raw) <= len(cities):
            return cities[int(raw) - 1]
        match = [c for c in cities if c.lower() == raw.lower()]
        if match:
            return match[0]
        print("Invalid choice, try again.")


def main():
    graph = build_graph(ROADS)
    cities = sorted(graph)
    print("=" * 50)
    print("  FOOTBALL MONK - AWAY-TRIP ROUTE PLANNER")
    print("  (Dijkstra shortest-path algorithm)")
    print("=" * 50)

    print("\nSelect your HOME city:")
    home = pick_city("Enter number or name: ", cities)
    dist, prev = dijkstra(graph, home)

    while True:
```

```
    print(f"\nHome city: {home}")
    print("1. Show shortest trip to ALL away stadiums")
    print("2. Plan a trip to ONE stadium")
    print("3. Change home city")
    print("4. Show MAP of all best routes from home")
    print("5. Save MAP of all best routes as PNG")
    print("6. Exit")
    choice = input("Choose: ").strip()

    if choice == "1":
        print_all_trips(home, dist, prev)
    elif choice == "2":
        dest = pick_city("Destination (number or name): ", cities)
        if dest == home:
            print("That is your home city - no travel needed!")
        else:
            print_one_trip(home, dest, dist, prev)
            if input("Show this route on the map? (y/n): ").strip().lower() == "y":
                draw_map(home, dist, prev, dest=dest)
    elif choice == "3":
        home = pick_city("New home city (number or name): ", cities)
        dist, prev = dijkstra(graph, home)
    elif choice == "4":
        draw_map(home, dist, prev)
    elif choice == "5":
        draw_map(home, dist, prev, save_as=f"routes_from_{home.lower()}.png")
    elif choice == "6":
        print("Enjoy the away day!")
        break
    else:
        print("Please enter 1-6.")


if __name__ == "__main__":
    main()
```

6.2 Explanation of the Code

- **build_graph(roads):** Converts the list of roads into an adjacency list; every road is stored in both directions because the graph is undirected.
- **dijkstra(graph, source):** Implements the algorithm with heapq. Stale heap entries (d > dist[u]) are skipped instead of being deleted, which is the standard lazy-deletion technique.
- **get_path(prev, target):** Walks the predecessor chain from the target back to the source and reverses it to give the route in travel order.
- **print_all_trips() and print_one_trip():** Print distance, travel time (distance divided by average speed), estimated cost (distance multiplied by cost per km) and the route.
- **pick_city() and main():** Accept a city by number or by name (case-insensitive), repeat on invalid input and provide the four-option menu.

6.3 Step-by-Step Trace (Dry Run)

The table below shows the order in which Dijkstra's algorithm settles the cities when the home city is Nottingham, together with the distances updated while each city's roads are relaxed.

Step	City settled	Distance (km)	Distances updated (relaxation)
1	Nottingham	0	Birmingham=80, Leicester=45, Sheffield=55
2	Leicester	45	London=205
3	Sheffield	55	Manchester=115, Leeds=110
4	Birmingham	80	Bristol=220, Wolverhampton=105
5	Wolverhampton	105	-
6	Leeds	110	Newcastle=270

Step	City settled	Distance (km)	Distances updated (relaxation)
7	Manchester	115	Liverpool=170
8	Liverpool	170	-
9	London	205	Brighton=290, Southampton=325
10	Bristol	220	-
11	Newcastle	270	Sunderland=290
12	Brighton	290	-
13	Sunderland	290	-
14	Southampton	325	Bournemouth=370
15	Bournemouth	370	-

Table 8. Step-by-step trace of Dijkstra's algorithm from Nottingham.

7. OUTPUT AND RESULTS

7.1 Application Output

For the home city Nottingham, option 1 of the menu produces the table of away trips below, sorted from the nearest to the farthest stadium. Time is calculated at an average speed of 70 km/h and cost at 0.12 per km.

Club	Stadium	km	Time	Cost	Shortest route
Leicester City	King Power Stadium	45	0h39m	5.40	Nottingham > Leicester
Sheffield United	Bramall Lane	55	0h47m	6.60	Nottingham > Sheffield
Aston Villa	Villa Park	80	1h09m	9.60	Nottingham > Birmingham
Wolves	Molineux	105	1h30m	12.60	Nottingham > Birmingham > Wolverhampton
Leeds United	Elland Road	110	1h34m	13.20	Nottingham > Sheffield > Leeds
Manchester United	Old Trafford	115	1h39m	13.80	Nottingham > Sheffield > Manchester

Club	Stadium	km	Time	Cost	Shortest route
Liverpool	Anfield	170	2h26m	20.40	Nottingham > Sheffield > Manchester > Liverpool
Arsenal	Emirates Stadium	205	2h56m	24.60	Nottingham > Leicester > London
Bristol City	Ashton Gate	220	3h09m	26.40	Nottingham > Birmingham > Bristol
Newcastle United	St James's Park	270	3h51m	32.40	Nottingham > Sheffield > Leeds > Newcastle
Brighton	Amex Stadium	290	4h09m	34.80	Nottingham > Leicester > London > Brighton
Sunderland	Stadium of Light	290	4h09m	34.80	Nottingham > Sheffield > Leeds > Newcastle > Sunderland
Southampton	St Mary's Stadium	325	4h39m	39.00	Nottingham > Leicester > London > Southampton
Bournemouth	Vitality Stadium	370	5h17m	44.40	Nottingham > Leicester > London > Southampton > Bournemouth

Table 9. Shortest away trips from Nottingham.

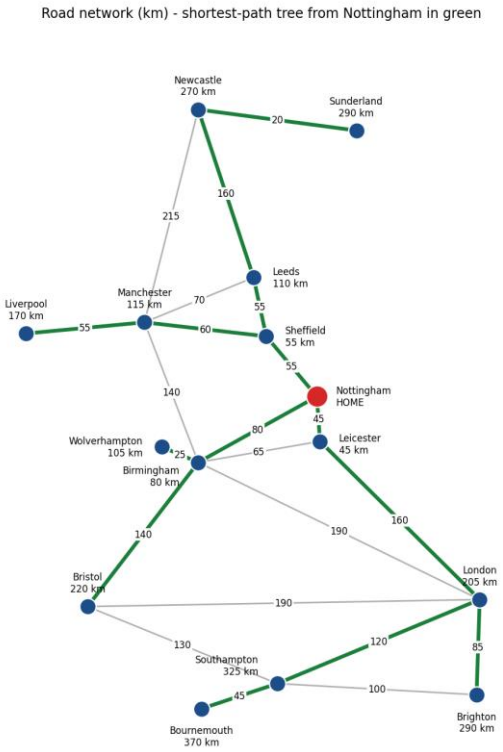


FIGURE 3 — SHORTEST-PATH TREE FROM NOTTINGHAM (GREEN EDGES) ON THE SAMPLE NETWORK

7.2 Result Analysis

The results show that the nearest away stadiums are Leicester (45 km) and Sheffield (55 km), while Bournemouth (370 km) is the farthest. The route to Newcastle is 270 km via Sheffield and Leeds, which is shorter than any other combination of roads in the network. Brighton and Sunderland are both 290 km away, so they appear as a tie in the sorted list. The tree of green edges in Figure 3 contains exactly $V - 1 = 14$ roads, because Dijkstra's predecessor links form a spanning tree rooted at the home city.

7.3 Test Cases and Validation

The following test cases were executed on the program to validate the algorithm and the user interface.

#	Test case	Input / action	Result
1	Correctness of distances	Run Dijkstra from every city and compare with Floyd–Warshall	All 225 of 225 source–destination distances matched
2	Menu: all trips	Home = Nottingham, option 1	14 stadiums listed, nearest to farthest (Table 9)

#	Test case	Input / action	Result
3	City entered by name	Type "nottingham" in lower case	Accepted (case-insensitive match)
4	Single stadium plan	Home = Nottingham, option 2, Newcastle	270 km, 3h 51m, cost 32.40, route via Sheffield and Leeds
5	Destination is the home city	Option 2 with the same city	Message "no travel needed" is shown
6	Invalid city input	Enter "zzz" and "99"	"Invalid choice, try again." shown twice, then input accepted
7	Invalid menu option	Enter 9 in the menu	"Please enter 1-4." and the menu is shown again
8	Change home city	Option 3, choose London, then option 1	Results are recalculated for London

Table 10. Test cases and validation.

Test 1 confirms the correctness of the implementation: the independent $O(V^3)$ Floyd–Warshall algorithm and the heap-based Dijkstra implementation gave identical distances for all 225 ordered pairs of the 15 cities.

8. FUTURE SCOPE AND FRAMEWORK ENHANCEMENTS

8.1 Real Data

Load road distances from a CSV or JSON file, or from a map API, instead of the built-in sample data.

8.2 Visualisation

Draw the network and highlight the chosen route with a graphical library such as matplotlib, networkx or Tkinter.

8.3 Fixture Integration

Read a season's fixture list and plan the total travel distance for all away matches of a club.

8.4 Faster Single-Target Search

Add A* search with straight-line distance as a heuristic for single-destination queries.

8.5 Multi-Stop Trips

Plan a tour that visits several stadiums, which leads to the travelling-salesman problem and to approximation algorithms.

8.6 Other Criteria

Use travel time or ticket price as edge weights, or combine them with distance.

9. CONCLUSION

The Away-Trip Route Planner meets its objectives. By modelling the road network as a weighted graph and applying Dijkstra's algorithm with a binary min-heap, the program finds the shortest distance and the exact route from the home city to every away stadium in O((V + E) log V) time. The output was validated against Floyd–Warshall on all city pairs, and the interactive menu makes the planner easy to use. The project helped in understanding the greedy strategy, relaxation, priority queues, lazy deletion and the practical meaning of time and space complexity.

References:

1. E. W. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, vol. 1, pp. 269–271, 1959.
2. T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 3rd ed., MIT Press, 2009.
3. R. Sedgewick and K. Wayne, Algorithms, 4th ed., Addison-Wesley, 2011.
4. J. Kleinberg and É. Tardos, Algorithm Design, Pearson, 2005.
5. Python Software Foundation, "heapq — Heap queue algorithm," Python 3 Documentation.
6. Project source code: away_trip_planner.py (Python 3).

10. LEARNING OUTCOMES

10.1 Graph Modelling

Representing a real road network as a weighted undirected graph using an adjacency list.

10.2 Greedy Strategy and Dijkstra's Algorithm

Understanding the greedy-choice property, relaxation, and why non-negative edge weights are required.

10.3 Priority Queues

Using a binary min-heap (heapq) with lazy deletion to select the nearest unsettled city in O(log V).

10.4 Complexity Analysis

Deriving O((V + E) log V) time and O(V + E) space, and comparing Dijkstra with BFS, Bellman–Ford, Floyd–Warshall and A*.

10.5 Validation and Testing

Verifying results against an independent algorithm and testing menu and user-input handling.

10.6 Python Programming

Modular functions, dictionaries and lists, and a console menu built with the standard library only.

APPENDIX

GitHub Repository: https://github.com/ashutoshdash8/football-monk-daa-project.git

